

Contents

	Page No
List of Figures	i
List of Tables	ii
Abbreviations	iii
Abstract	iv
1. Introduction	1-2
1.1 Introduction of the Project	1
1.2 Existing System	1
1.3 Problems in Existing System	1
1.4 Proposed System	1
1.5 Advantages of Proposed System	2
2. Requirements Analysis	3-7
2.1 Functional Requirements	4
2.2 Non-functional Requirements	5
2.3 Computational Resource Requirements	5
2.3.1 Hardware Requirements	5
2.3.2 Software Requirements	5
2.4 Life Cycle Model	6-7
3. Design	8-16
3.1 Architecture	9-10
3.1.1 Software Architecture	9
3.1.2 Technical Architecture	10
3.2 Use Case Diagram	11
3.3 Class Diagram	12
3.4 Sequence Diagram	13-14
3.5 Activity Diagram	14-15
3.6 Database	16

4.	Implementation	17-26
	4.1 Technologies	18-20
	4.1.1 Java	18
	4.1.2 Features of Java	18
	4.1.3 Spring Boot Framework	19
	4.1.4 Features of Spring Boot Framework	19-20
	4.2 Pseudocode	20-26
5.	Testing	27-31
	5.1 Overview	28
	5.2 Dimensions of Testing	28
	5.3 Stages of Testing	29-31
	5.3.1 Unit Testing	29
	5.3.2 Integration Testing	29
	5.3.3 System Testing	29
	5.3.4 Acceptance Testing	30
	5.4 Types of Testing	30
	5.5 Test Cases	31
6.	Screenshots	32-40
7.	Conclusion and Future Scope	41-42
	7.1 Conclusion	42
	7.2 Future Scope	42
8.	References	43-44

List of Figures

Fig. No	Figure Name	Page. No
2.1	Waterfall Model	7
3.1	Software Architecture	9
3.2	Technical Architecture	10
3.3	Use Case Diagram for User	11
3.4	Use Case Diagram for Admin	11
3.5	Class Diagram	12
3.6	Sequence Diagram for User	13
3.7	Sequence Diagram for Admin	14
3.8	Activity Diagram for User	15
3.9	Activity Diagram for Admin	15
6	Screenshots	32-40
	6.1 Main page	33
	6.2 User Registration	34
	6.3 User Login	35
	6.4 Uploading Syllabus Copy	36
	6.5 Syllabus Copy Preview	37
	6.6 Results Page	38
	6.7 Play Video	39
	6.8 Delete Video	40

List of Tables

Table. No	Table Name	Page. No
3.10	Users Table	16
5.1	Test Cases	31

Abbreviations

IDE	: Integrated Development Environment
UML	: Unified Modelling Language
OCR	: Optimal Character Recognition
WORA	: Write Once Run Anywhere
JVM	: Java Virtual Machine
REST	: Representational State Transfer
API	: Application Programming Interface

Abstract

The "StudyFlow" platform is a user-friendly web-based system designed to enhance students' learning experience by organizing educational content based on their syllabus. StudyFlow simplifies the learning process by allowing students to upload their syllabus documents, based on which it curates a list of relevant YouTube videos aligned with the topics. The platform eliminates the need for manual searching, saving students valuable time and effort while ensuring they access high-quality, syllabus-relevant content. By minimizing distractions from non-educational material and providing structured learning resources, StudyFlow empowers students to create efficient study plans. This innovative solution bridges the gap between generic video platforms and academic needs, promoting focused and effective learning. StudyFlow leverages technology to streamline education, making it a valuable tool for students to achieve their academic goals.

1. INTRODUCTION

1. Introduction

1.1 Introduction of the Project

StudyFlow is a platform designed to help students organize videos into topic-based groups for easy access. It ensures complete coverage of syllabus topics, unlike YouTube playlists, which often miss key areas. By providing all relevant content in one place, StudyFlow saves students time and effort. The platform allows students to create structured study plans around curated playlists, improving time management. With efficient organization, it eliminates the hassle of searching for resources. StudyFlow ensures a distraction-free, tailored learning experience. Overall, StudyFlow makes learning more organized, accessible, and efficient for students.

1.2 Existing System

The existing system relies on platforms like YouTube and Vimeo for educational content. YouTube offers a wide range of resources with videos by educators that simplify complex topics, helping students learn effectively. Vimeo focuses on professional content with advanced tools like video production and analytics, catering to creators seeking polished presentations. However, YouTube playlists often miss specific syllabus topics, and Vimeo is less academic-focused. This creates a gap, highlighting the need for a platform like StudyFlow, which organizes videos by syllabus and topics for efficient and tailored learning.

1.3 Problems in Existing System

- Platforms like YouTube often feature unrelated videos, diverting students' attention from their studies.
- Searching for relevant videos wastes valuable study time, reducing overall productivity.
- Students may be exposed to inappropriate content, requiring additional supervision from educators or parents.

1.4 Proposed System

The proposed system introduces a simpler and more reliable approach to organizing educational content, making it user-friendly and efficient. Users can upload images of their syllabus in various formats such as .PNG, .JPG, etc., and the system will automatically generate a curated list of YouTube videos that are directly relevant to the topics outlined in the syllabus. The system streamlines the process, making learning more structured and accessible, saving time and effort for students.

1.5 Advantages of Proposed System

- The system automatically generates curated lists, saving students time and effort in finding the right videos.
- Content is curated based on the specific topics in the syllabus, ensuring relevance.
- The system focuses solely on educational videos, eliminating unrelated content.

2. REQUIREMENT ANALYSIS

2. Requirement Analysis

2.1 Functional Requirements

StudyFlow is a web-based application designed to help students organize educational videos based on their syllabus topics. The system allows students to easily access relevant content, enhancing their learning experience.

StudyFlow consists of the following modules:

1. User Module
2. Admin Module

1. User Module:

The user (student) can register, log in, upload their syllabus, view and play the curated list of educational videos, and log out from the system.

- Register
- Login
- View Dashboard
- Upload Syllabus Copy
- View Curated Videos List
- Play Video
- Delete Video
- Logout

2. Admin Module:

The admin is responsible for managing user accounts, overseeing the system's operations, monitoring user performance, and ensuring the proper functioning of the platform.

- Login

- View User Accounts
- Delete User Accounts
- Monitor Performance
- Logout

2.2 Non - Functional Requirements

- Scalability: The system should be able to handle an increasing number of users and a growing volume of educational content without performance degradation.
- Usability: The platform should provide a user-friendly interface for students and admin.
- Maintainability: The system should have well-structured, modular, and easily maintainable code to facilitate future updates, bug fixes, and feature additions.
- Performance: The system should deliver quick content loading, low latency, and smooth video playback, ensuring efficient content delivery even under high user loads.
- Security: The platform should implement robust security protocols and data protection to safeguard user information and prevent unauthorized access.
- Reliability: The system should ensure high availability, with minimal downtime, and have recovery mechanisms to prevent data loss in case of system failure.

2.3 Computational Resource Requirements

2.3.1 Hardware Requirements

- Processor : Intel Processor
- RAM : 4GB
- Hard Disk : 40GB

2.3.2 Software Requirements

- Operating System : Windows 10
- Front-end Framework : React JS
- Back-end Framework : Spring Boot
- Database : MySQL

- IDE : Visual Studio Code, IntelliJ Idea

2.4 Life Cycle Model

The Waterfall Model is a linear and sequential software development process, where each phase must be completed before the next one begins. This model is best suited for projects where requirements are well understood and unlikely to change. It consists of the following phases:

- Requirement Gathering & Analysis: Define the project goals and collect detailed requirements from stakeholders. This phase focuses on understanding what the system needs to achieve. The documented requirements will guide the entire project.
- System Design: Create the system architecture and design detailed components based on gathered requirements. It involves both high-level and low-level design to map out the structure, data flow, and component interactions.
- Implementation (Development): Write the code and build the system as per the design specifications. Developers create and integrate modules to form the functional system.
- Testing: Verify the system's functionality, performance, and reliability. Various testing types are performed to ensure the system meets the requirements and works as intended.
- Deployment: Release the system into the production environment for user access. It involves installation, configuration, and ensuring the system operates correctly in the live environment.
- Maintenance: Monitor the system and perform updates as needed. This phase includes bug fixes, performance improvements, and adding new features based on user feedback.

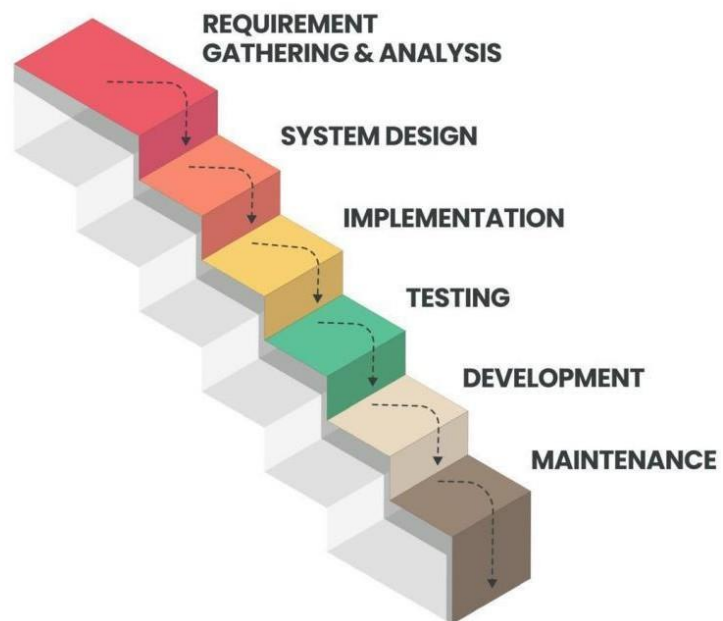


Fig 2.1 Waterfall Model

3. DESIGN

3. Design

3.1 Architecture

Project architecture represents number of components we are using as a part of our project and the flow of request processing i.e. what components in processing the request and in which order. An architecture description is a formal description and representation of a system organized in a way that supports reasoning about the structure of the system. Architecture is of two types. They are:

- (1) Software Architecture
- (2) Technical Architecture

3.1.1 Software Architecture

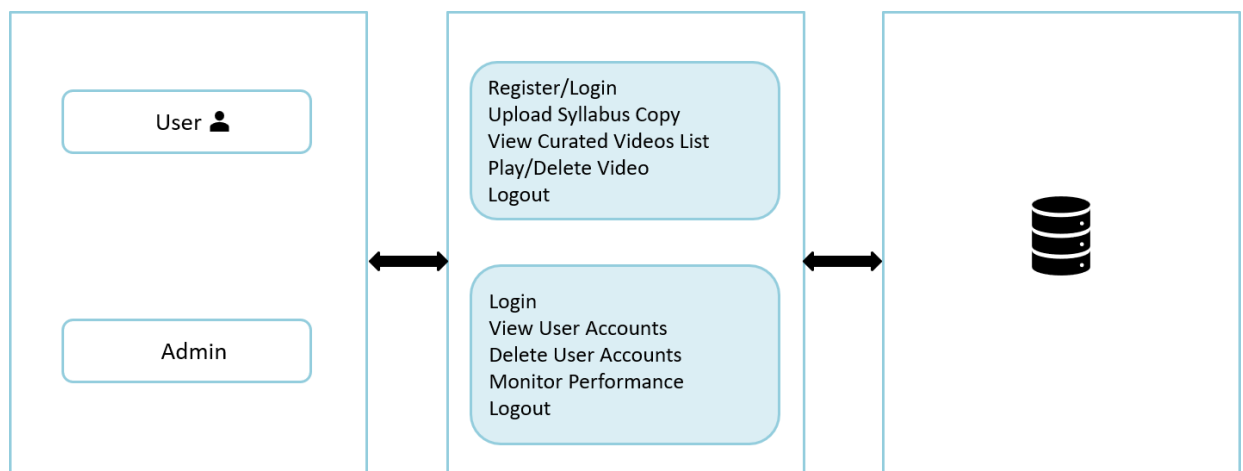


Fig 3.1 Software architecture

3.1.2 Technical Architecture:

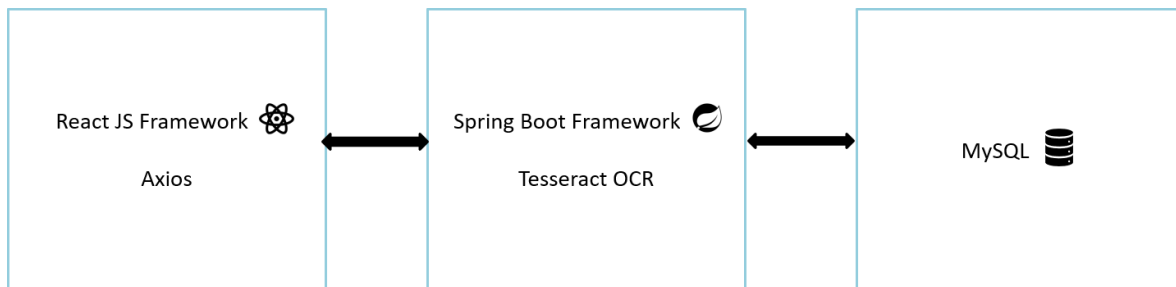


Fig 3.2 Technical architecture

UML Diagrams

Design engineering deals with the Unified Modelling Language (UML) which is a standard language for writing software blue prints. The UML is a language for

- Visualizing
- Specifying
- Constructing
- Documenting the artifacts of a software intensive system

The UML is a language which provides vocabulary and the rules for the rules combining words in that vocabulary for the purpose of communication. A modelling language is a language whose vocabulary and the rules focus on the conceptual and physical representation of a system. Modelling yields an understanding of a system.

3.2 Use Case Diagram:

Use case diagram are used to gather the requirements of a system including internal and external influences. A use case represents a functionality of a system. So, use case diagrams are used to describes the relationships among the functionalities and their internal/external controllers. These controllers are known as actors.

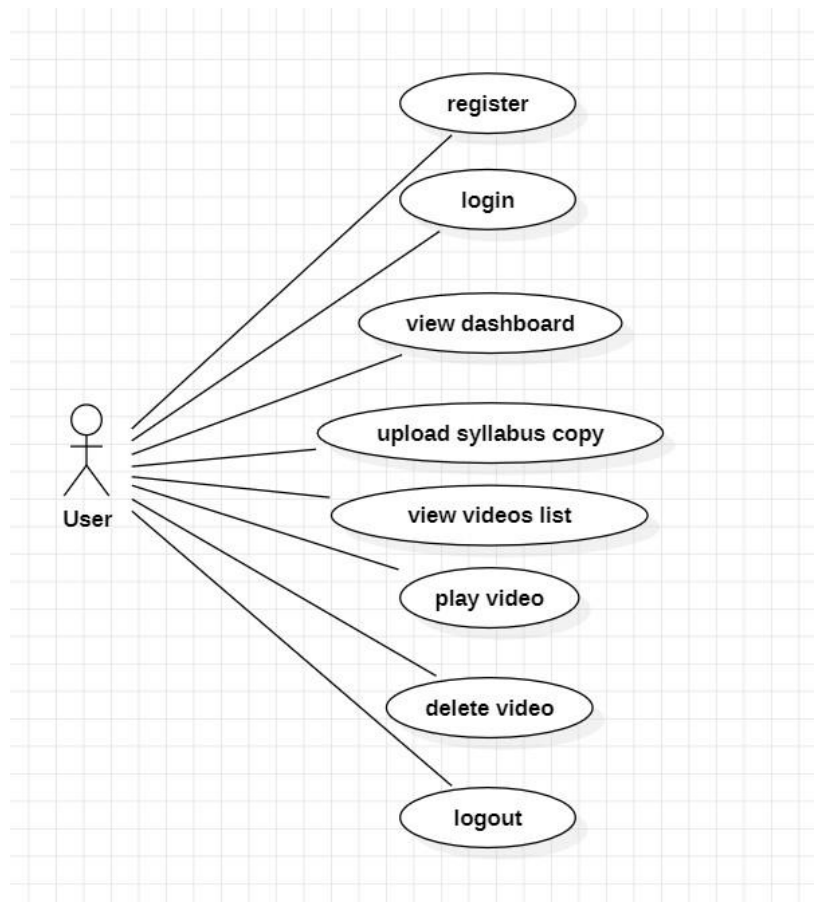


Fig 3.3 Use case diagram for user

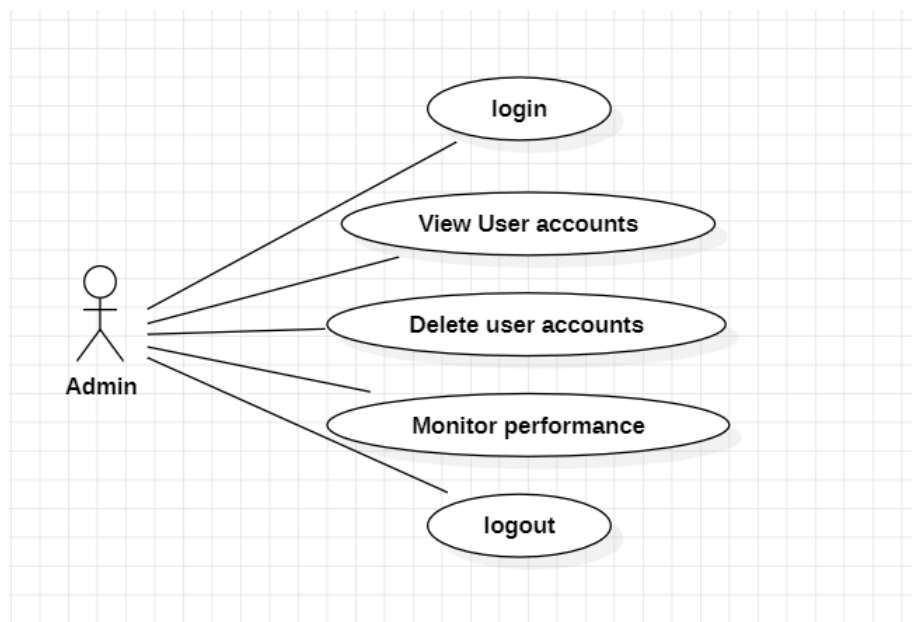


Fig 3.4 Use case diagram for admin

3.3 Class Diagram:

A class diagram in the Unified Modelling Language is a type of a static structure diagram that describes the structure of a system showing the system's classes, their attributes, operations (or methods), and the relationship among the classes.

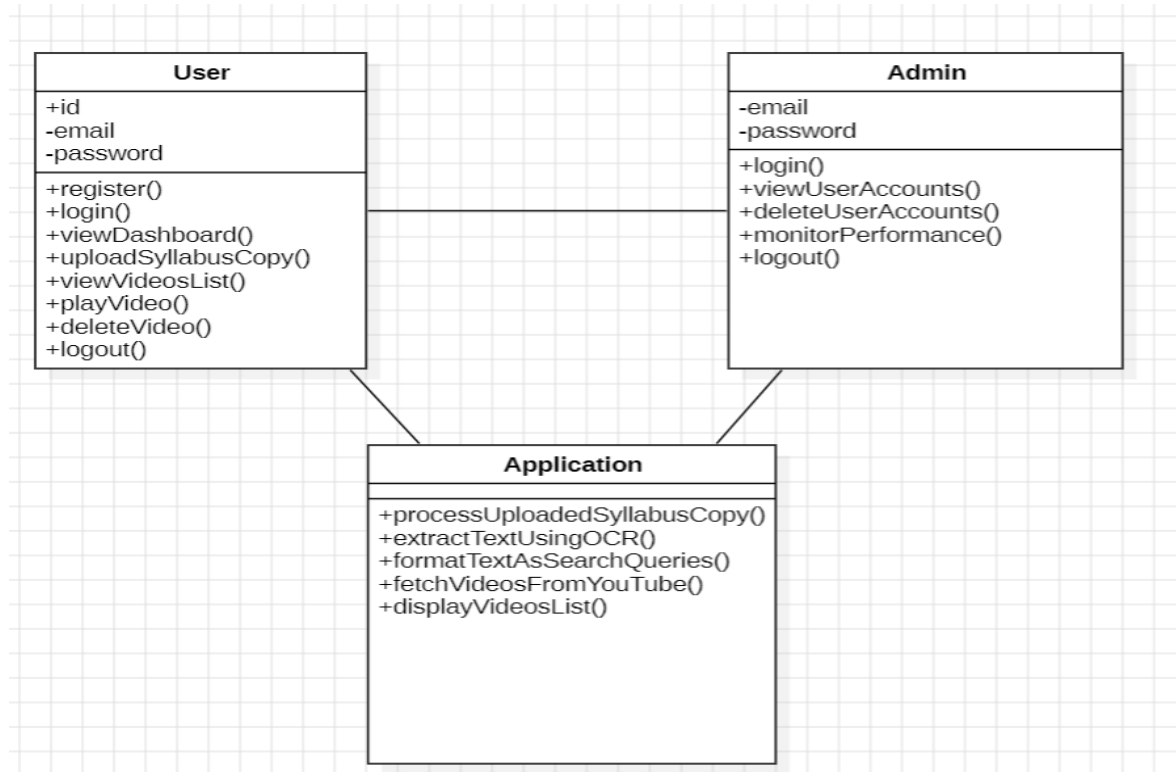


Fig 3.5 Class Diagram

3.4 Sequence Diagram:

A sequence is Unified Modelling Language (UML) is a kind of interaction diagram that shows how processes operate with another and in what order. It is a construct of a message chart.

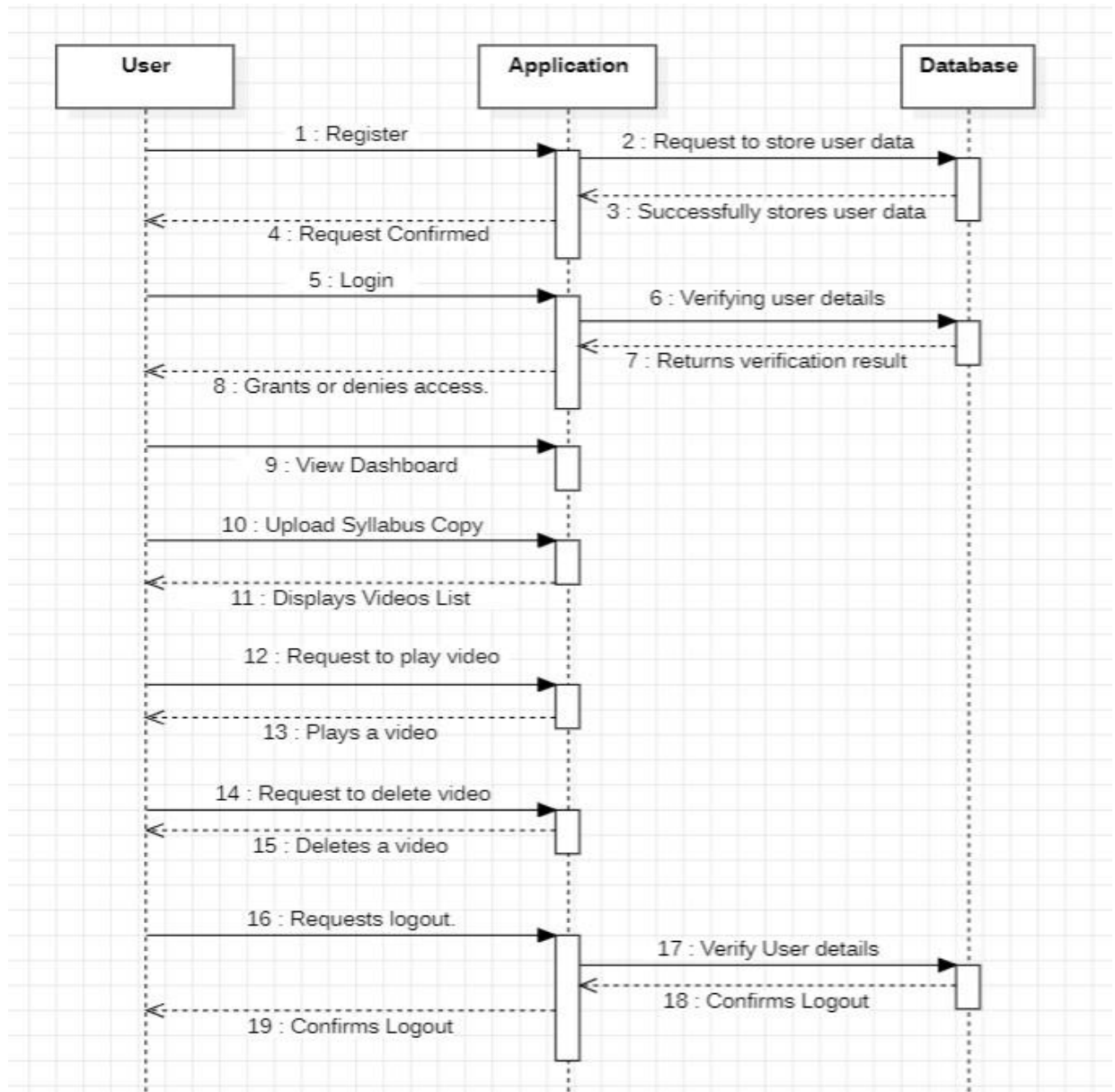


Fig 3.6 Sequence diagram for User

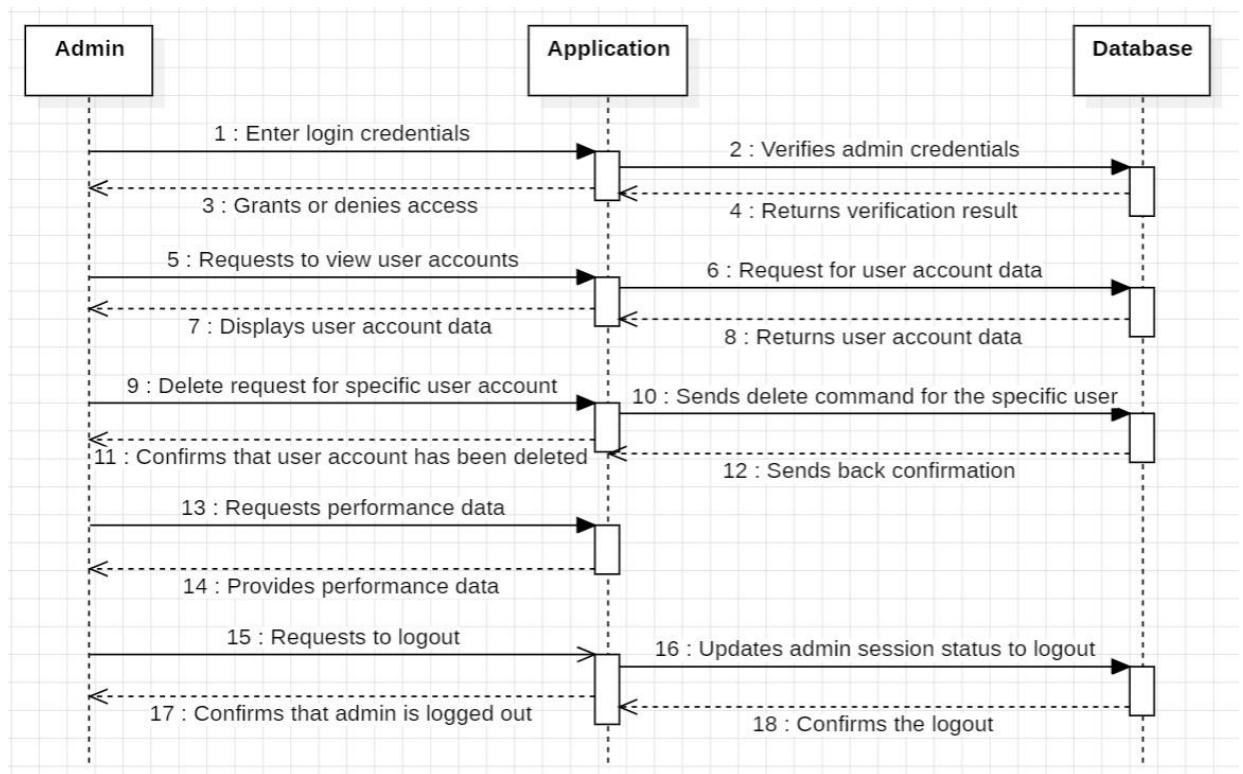


Fig 3.7 Sequence diagram for Admin

3.5 Activity Diagram:

Activity diagrams are graphical representation of workflows of stepwise activities with support for choice, iteration and concurrency. In the Unified Modelling activity diagram can be used to describe the business and operational workflows of components in a system. An activity diagram shows the overall control.

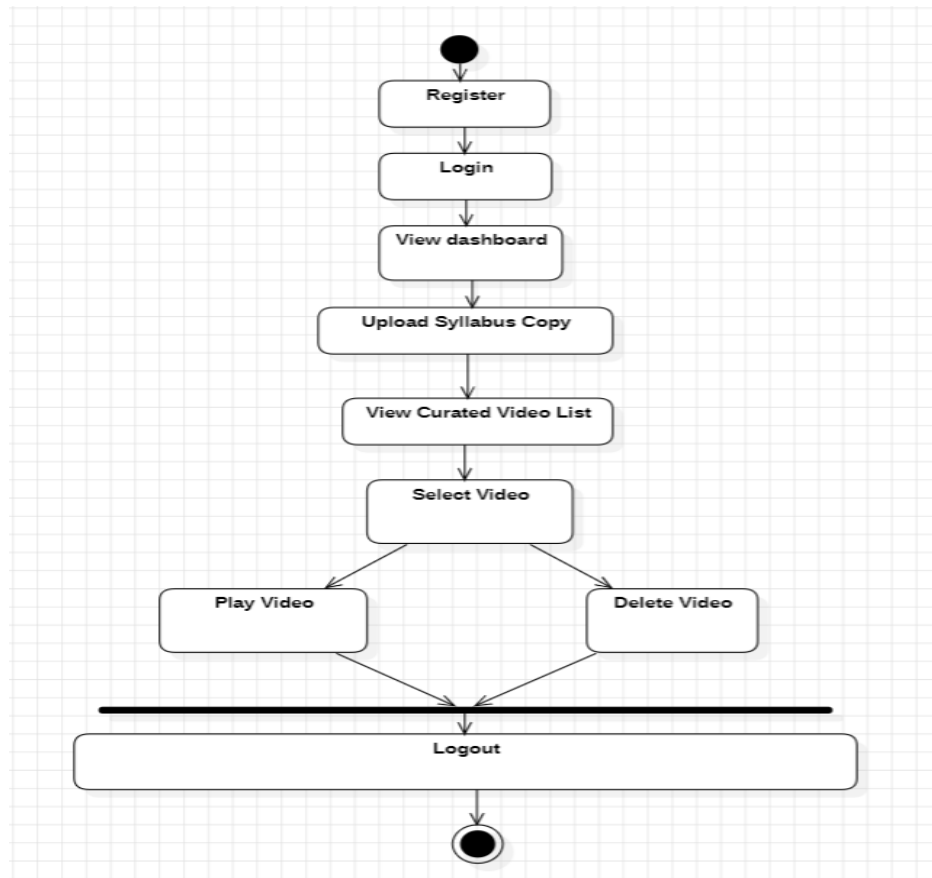


Fig 3.8 Activity diagram for User

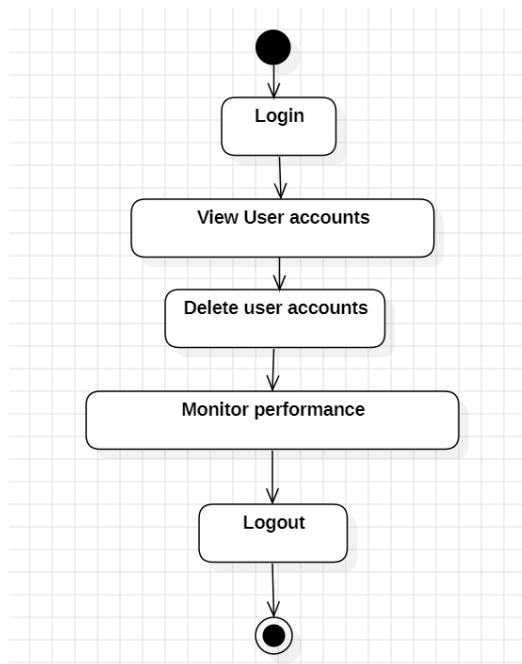
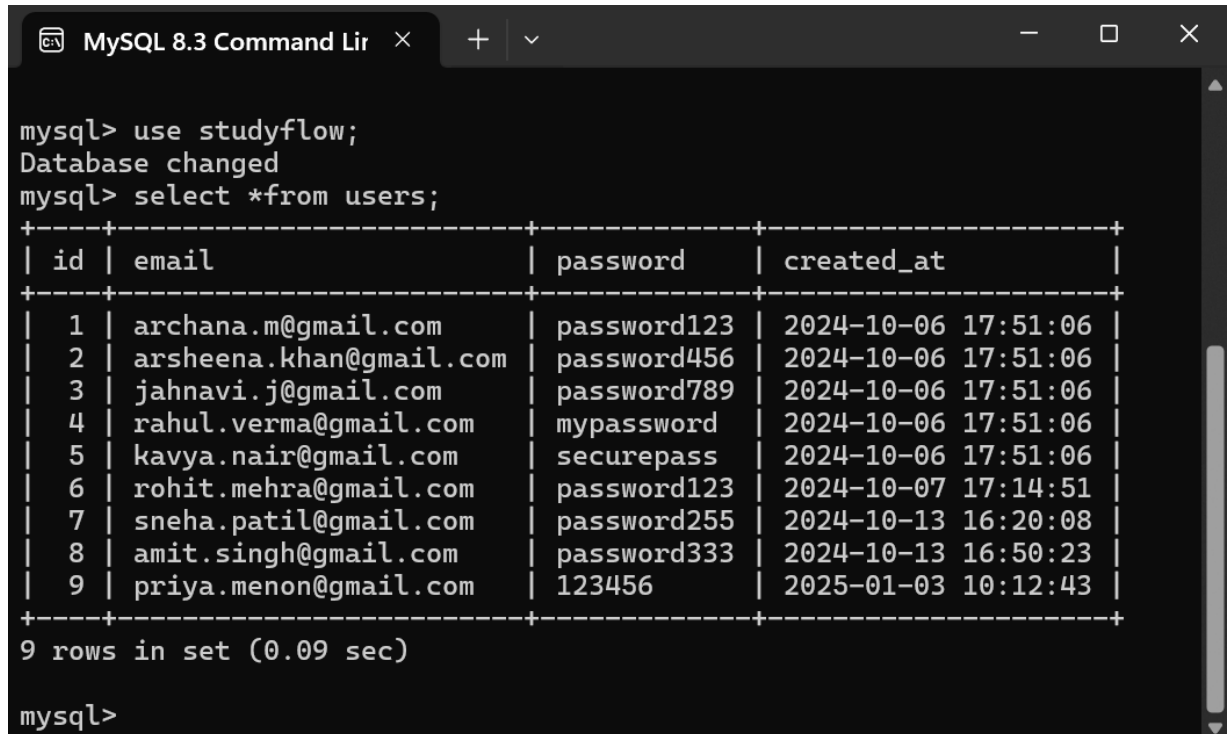


Fig 3.9 Activity diagram for Admin

3.6 Database

Table 3.10 Users Table



The screenshot shows a MySQL 8.3 Command Line Interface window. The user has entered the command `mysql> use studyflow;` and the output is `Database changed`. Then, the user entered `mysql> select *from users;` and the output is a table with 9 rows. The table has columns `id`, `email`, `password`, and `created_at`. The data is as follows:

id	email	password	created_at
1	archana.m@gmail.com	password123	2024-10-06 17:51:06
2	arsheena.khan@gmail.com	password456	2024-10-06 17:51:06
3	jahnavi.j@gmail.com	password789	2024-10-06 17:51:06
4	rahul.verma@gmail.com	mypassword	2024-10-06 17:51:06
5	kavya.nair@gmail.com	securepass	2024-10-06 17:51:06
6	rohit.mehra@gmail.com	password123	2024-10-07 17:14:51
7	sneha.patil@gmail.com	password255	2024-10-13 16:20:08
8	amit.singh@gmail.com	password333	2024-10-13 16:50:23
9	priya.menon@gmail.com	123456	2025-01-03 10:12:43

9 rows in set (0.09 sec)

mysql>

4. IMPLEMENTATION

4.Implementation

4.1 Technologies

This system is developed using Java programming language using Spring Boot Framework.

4.1.1 Java

Java is a versatile and widely-used high-level programming language developed by Sun Microsystems (now part of Oracle) in 1995. Known as a robust, object-oriented, and secure language, Java was originally named Oak but was renamed due to trademark conflicts. Designed by James Gosling, Java enables "Write Once, Run Anywhere" (WORA), making it platform-independent. Programs written in Java are compiled to bytecode, executable on any Java Virtual Machine (JVM), regardless of the underlying architecture.

Java simplifies programming with a C++-like syntax while eliminating complexities such as explicit pointers and operator overloading. It also features automatic garbage collection for memory management. Java is used in diverse domains, from desktop applications to scientific computing, mobile devices, and web servers. Its popularity stems from its ease of learning, high performance, security, and strong community support.

4.1.2 Features of Java

- Simple: Easy-to-understand syntax based on C++ with removed complexities. Java syntax is based on C++ (so easier for programmers to learn it after C++).
- Object-Oriented: Everything in Java revolves around objects, utilizing principles like inheritance, polymorphism, abstraction, and encapsulation.
- Platform Independent: Java bytecode can run on any platform with a JVM.
- Security: Eliminates explicit pointers, and runs in a virtual machine sandbox.
- Robust: Features strong memory management and exception handling.

- **Portable:** Java bytecode can run on multiple platforms without modification.
- **High Performance:** Faster than interpreted languages but slower than compiled languages like C++.
- **Multi-Threading:** Supports concurrent task execution with shared memory space.

4.1.3 Spring Boot Framework

Spring Boot is a powerful Java-based framework designed to simplify the development of Spring applications. It eliminates the need for extensive configuration by providing intelligent defaults and auto-configuration features, making it easy to build stand-alone, production-ready applications. With Spring Boot, developers can focus more on business logic rather than dealing with complex setups and configurations.

One of the standout features of Spring Boot is its ability to embed web servers like Tomcat, Jetty, or Undertow, allowing applications to run independently without requiring external deployment. The framework also offers "starter dependencies," which bundle commonly used libraries and tools for specific functionalities, such as Spring Web, Spring Security, and Spring Data. This approach minimizes dependency management complexity and speeds up development.

Additionally, Spring Boot is ideal for building microservices due to its lightweight and modular design. It includes built-in production-ready features like metrics, health checks, and externalized configuration, which simplify monitoring and scaling applications. Overall, Spring Boot streamlines the process of creating robust and scalable Java applications, making it a popular choice among developers.

4.1.4 Features of Spring Boot Framework

- **Simplified Configuration:** Automatic setup using annotations and application.properties or application.yml files.

- **Microservices Ready:** Perfectly suited for building microservices architectures due to its lightweight nature.
- **Dependency Management:** Offers a curated list of dependencies, reducing compatibility issues.
- **Spring Ecosystem Integration:** Seamlessly integrates with Spring Framework modules like Spring MVC, Spring Data, and Spring Security.
- **REST API Development:** Simplifies the creation of RESTful APIs with minimal configuration.

4.2 Pseudocode

User Registration and Login Service

```
package com.studyflow.user_login_register_service.service;
```

```
import com.studyflow.user_login_register_service.model.User;
import com.studyflow.user_login_register_service.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.Optional;
```

```
@Service
```

```
public class UserService {
```

```
    @Autowired
```

```
    private UserRepository userRepository;
```

```
    // Registration logic
```

```
    public User register(User user) {
```

```
        // Check if user already exists by email
```

```
        Optional<User> existingUser = userRepository.findByEmail(user.getEmail());
```

```
        if (existingUser.isPresent()) {
```

```
        throw new RuntimeException("User already exists with email: " +
user.getEmail());
    }
    return userRepository.save(user);
}
// Login logic
public Optional<User> login(String email, String password) {
    return userRepository.findByEmailAndPassword(email, password);
}
}
```

OCR Service

```
package com.studyflow.ocr_service.service;

import net.sourceforge.tess4j.Tesseract;
import net.sourceforge.tess4j.TesseractException;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import java.io.File;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class OcrService {
    public List<String> parseImage(MultipartFile imageFile) throws IOException,
TesseractException {
        // Save uploaded image to a temporary file
        File convFile = new File(System.getProperty("java.io.tmpdir") + "/" +
imageFile.getOriginalFilename());
        imageFile.transferTo(convFile);
        System.out.println("Temporary file saved at: " + convFile.getAbsolutePath());
    }
}
```

```
// Initialize Tesseract instance
Tesseract tesseract = new Tesseract();
tesseract.setDatapath("C:\\Program Files\\Tesseract-OCR\\tessdata");
tesseract.setLanguage("eng"); // Set language to English, change as necessary

// Perform OCR on the image file
String parsedText = tesseract.doOCR(convFile);
System.out.println("Parsed text: " + parsedText);

// Replace newlines preceded or followed by alphanumeric characters or punctuation
with a space
parsedText = parsedText.replaceAll("(?<=[A-Za-z0-9,;])\\n(?:=[A-Za-z0-9,;])", " ");
System.out.println("Processed text after newline handling: " + parsedText);

// Split text into a list based on commas and periods
List<String> topics = Arrays.asList(parsedText.split("[,\\.]"));

// Capitalize the first letter of each topic and remove extra whitespace,
// while preserving the original case of the rest of the text
topics = topics.stream()
    .map(topic -> topic.trim())
    .filter(topic -> !topic.isEmpty()) // Remove any empty topics
    .map(topic -> capitalizeFirstLetterOnly(topic)) // Capitalize first letter, preserve
the rest
    .collect(Collectors.toList());

System.out.println("Extracted topics: " + topics);
return topics;
}

// Helper method to capitalize only the first letter of a string while preserving the rest
private String capitalizeFirstLetterOnly(String text) {
```

```
        if (text == null || text.isEmpty()) {  
            return text;  
        }  
        return text.substring(0, 1).toUpperCase() + text.substring(1); // Only change the first  
letter  
    }  
}
```

YouTube Service

```
package com.studyflow.youtube_service.service;  
  
import com.studyflow.youtube_service.config.YouTubeConfig;  
import com.studyflow.youtube_service.model.VideoResult;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Service;  
import org.springframework.web.client.RestTemplate;  
import org.json.JSONArray;  
import org.json.JSONObject;  
import java.util.ArrayList;  
import java.util.List;  
  
@Service  
public class YouTubeService {  
    private static final String YOUTUBE_SEARCH_API =  
"https://www.googleapis.com/youtube/v3/search";  
    private static final String YOUTUBE_VIDEO_API =  
"https://www.googleapis.com/youtube/v3/videos";  
  
    @Autowired  
    private YouTubeConfig config;  
    /**  
    * Fetch YouTube videos based on topic, subject, and optional channel.  
    * Returns the top 3 videos with channel name and view count.
```

```
*/

public List<VideoResult> fetchYouTubeVideos(String topic, String subjectName, String
channelName) {
    String searchQuery = buildSearchQuery(topic, subjectName);
    List<VideoResult> videoResults = new ArrayList<>();
    try {
        String url = YOUTUBE_SEARCH_API + "?part=snippet&q=" + searchQuery +
"&type=video&maxResults=3&key=" + config.getApiKey();

        // If the user provides a channel, add channelId filter
        if (channelName != null && !channelName.isEmpty()) {
            String channelId = fetchChannelId(channelName);
            if (channelId != null) {
                url += "&channelId=" + channelId;
            }
        }

        RestTemplate restTemplate = new RestTemplate();
        String response = restTemplate.getForObject(url, String.class);
        JSONObject jsonResponse = new JSONObject(response);
        JSONArray items = jsonResponse.getJSONArray("items");
        for (int i = 0; i < items.length(); i++) {
            JSONObject item = items.getJSONObject(i);
            JSONObject snippet = item.getJSONObject("snippet");
            String videoId = item.getJSONObject("id").getString("videoId");
            String title = snippet.getString("title");
            String channelTitle = snippet.getString("channelTitle");
            // Get view count
            long viewCount = getVideoViewCount(videoId);
            videoResults.add(new VideoResult(title, videoId, channelTitle, viewCount));
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

```
    }  
    return videoResults;  
}  
/**  
 * Builds the search query using the topic and subject.  
 */  
private String buildSearchQuery(String topic, String subjectName) {  
    StringBuilder query = new StringBuilder();  
    query.append(topic);  
  
    if (subjectName != null && !subjectName.isEmpty()) {  
        query.append(" ").append(subjectName);  
    }  
    return query.toString();  
}  
/**  
 * Fetches channel ID based on the channel name.  
 */  
private String fetchChannelId(String channelName) {  
    String url = YOUTUBE_SEARCH_API + "?part=snippet&q=" + channelName +  
"&type=channel&key=" + config.getApiKey();  
    RestTemplate restTemplate = new RestTemplate();  
    String response = restTemplate.getForObject(url, String.class);  
    JSONObject jsonResponse = new JSONObject(response);  
    JSONArray items = jsonResponse.getJSONArray("items");  
    if (items.length() > 0) {  
        return items.getJSONObject(0).getJSONObject("id").getString("channelId");  
    }  
    return null; // No channel found  
}
```



```
/**
 * Fetches view count for a given video ID.
 */
private long getVideoViewCount(String videoId) {
    String url = YOUTUBE_VIDEO_API + "?part=statistics&id=" + videoId + "&key=" +
    config.getApiKey();
    RestTemplate restTemplate = new RestTemplate();
    String response = restTemplate.getForObject(url, String.class);
    JSONObject jsonResponse = new JSONObject(response);
    JSONArray items = jsonResponse.getJSONArray("items");
    if (items.length() > 0) {
        return items.getJSONObject(0).getJSONObject("statistics").getLong("viewCount");
    }
    return 0; // Default view count if not found
}
}
```

5. TESTING

5. Testing

5.1 Overview

Software testing is a process, to evaluate the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and to identify the defects to ensure that the product is defect free in order to produce the quality product.

As per the current trend, due to constant change and development in digitization, our lives are improving in all areas. The way we work is also changed. We access our bank online; we do shop online; we order food online and many more. We rely on software's and systems. What if these systems turnout to be defective? We all know that one small bug shows huge impact on business in terms of financial loss and goodwill. To deliver a quality product, we need to have Software Testing in the Software Development Process.

Some of the reasons why software testing becomes very significant and integral part in the field of information technology are as follows.

1. Cost effectiveness
2. Customer Satisfaction
3. Security
4. Product Quality

5.2 Dimensions of Testing

There are many different dimensions to consider:

1. Layers of the application (database, APIs, UI)
2. Scale of testing (unit, module, integration, scenario)
3. Type of testing (functional, performance, security, etc.)
4. Methodology (exploratory, scripted manual, automated)

5.3 Stages of Testing

5.3.1 Unit Testing

For StudyFlow, unit testing focuses on specific components like the OCR processing module, YouTube Data API integration, and user authentication. Each function, such as parsing text into topics or saving user preferences, is tested individually to ensure it works as expected. White box testing is commonly used to validate these components, checking the internal logic and code paths. A significant benefit is the ability to re-run these tests after every code change, enabling early detection and resolution of bugs. For example, when modifying the OCR parsing logic, unit tests ensure that the text is still correctly converted into topics without introducing errors.

5.3.2 Integration Testing

Integration testing for StudyFlow combines various units, such as the OCR module, YouTube API module, and the React frontend, to test them as a cohesive group. The objective is to identify any interface defects between these modules. For instance, integration tests ensure that the topics extracted by the OCR module are passed correctly to the API, and the resulting video list is displayed seamlessly on the frontend. Tools like Postman and automated scripts help test APIs and inter-module communication to ensure smooth functionality.

5.3.3 System Testing

System testing evaluates the entire StudyFlow application as a whole to ensure it meets user requirements and quality standards. This includes testing features such as syllabus image upload, text-to-topic conversion, video retrieval, and user login. Conducted in an environment resembling production, independent testers simulate real-world scenarios to validate that the application behaves as intended. For instance, they test uploading images of various sizes and formats to confirm compliance with the system's constraints while verifying seamless user navigation and performance.

5.3.4 Acceptance Testing

Acceptance testing determines whether StudyFlow meets the end users' needs and is ready for release. Users test the application by uploading syllabus images and verifying that accurate topics and corresponding videos are displayed. This phase ensures the software aligns with educational requirements and usability expectations. For example, during acceptance testing, users check if saved videos are retrievable and confirm that the topic-wise playlist structure enhances their learning experience. Only after passing these tests is StudyFlow deployed to production.

5.4 Types of Testing

5.4.1 Black box testing

In StudyFlow, black box testing is employed to evaluate the system's functionality without delving into its internal code structure. Testers assess the application's behavior by providing inputs like syllabus images or user credentials and observing the outputs. For instance, testers validate that uploading a valid image correctly extracts topics and displays relevant YouTube videos, ensuring no knowledge of the OCR or API integration logic is required.

5.4.2 White box testing

White box testing is applied to StudyFlow to verify the internal workings of critical components, such as the OCR text parsing logic and API request/response handling. This approach involves examining the code to design test cases, ensuring proper coverage of all possible scenarios.

White Box Testing Techniques:

1. Statement Coverage
2. Branch Coverage
3. Path Coverage

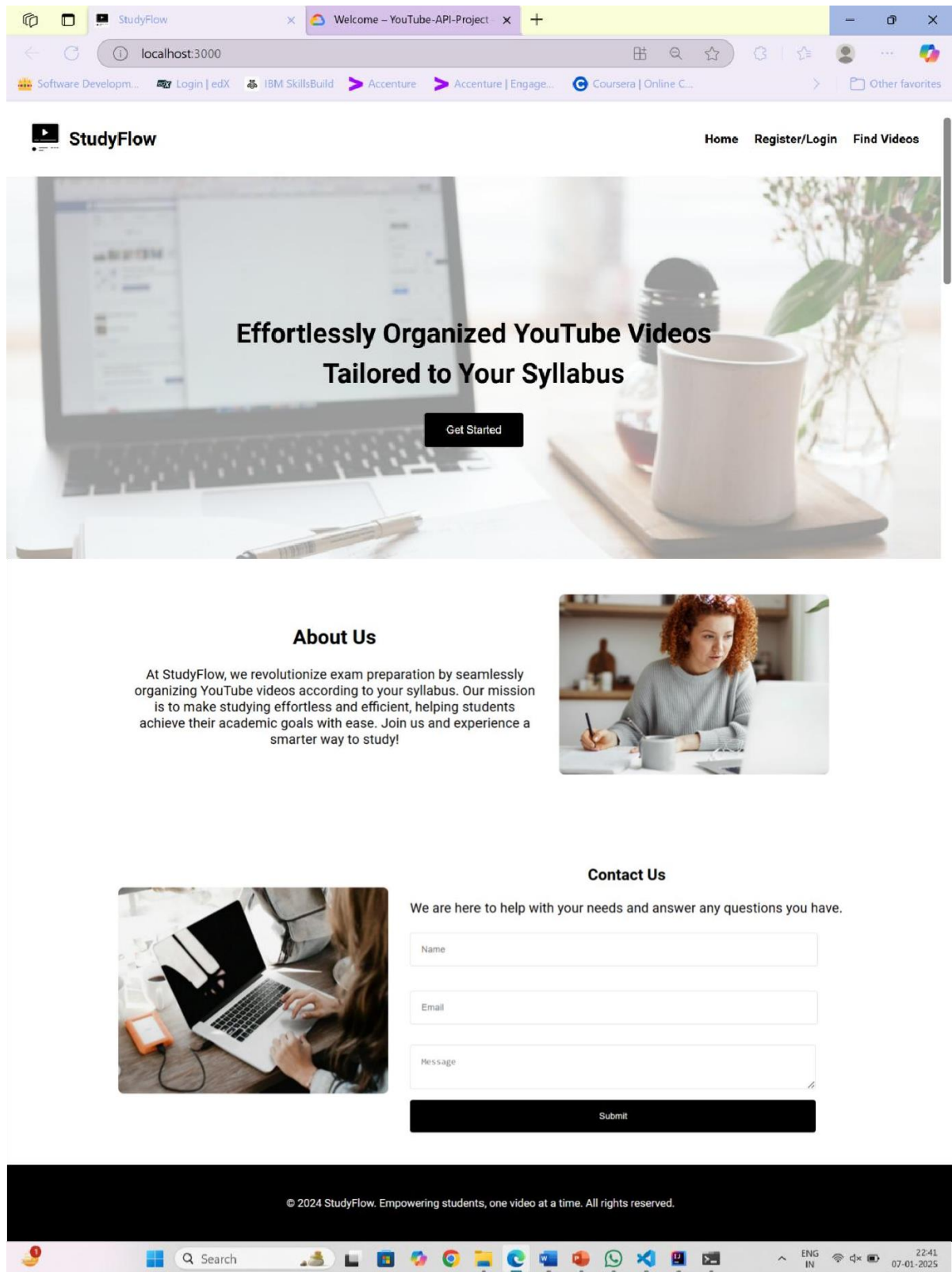
5.5 Test Cases

Table 5.1 Test Cases

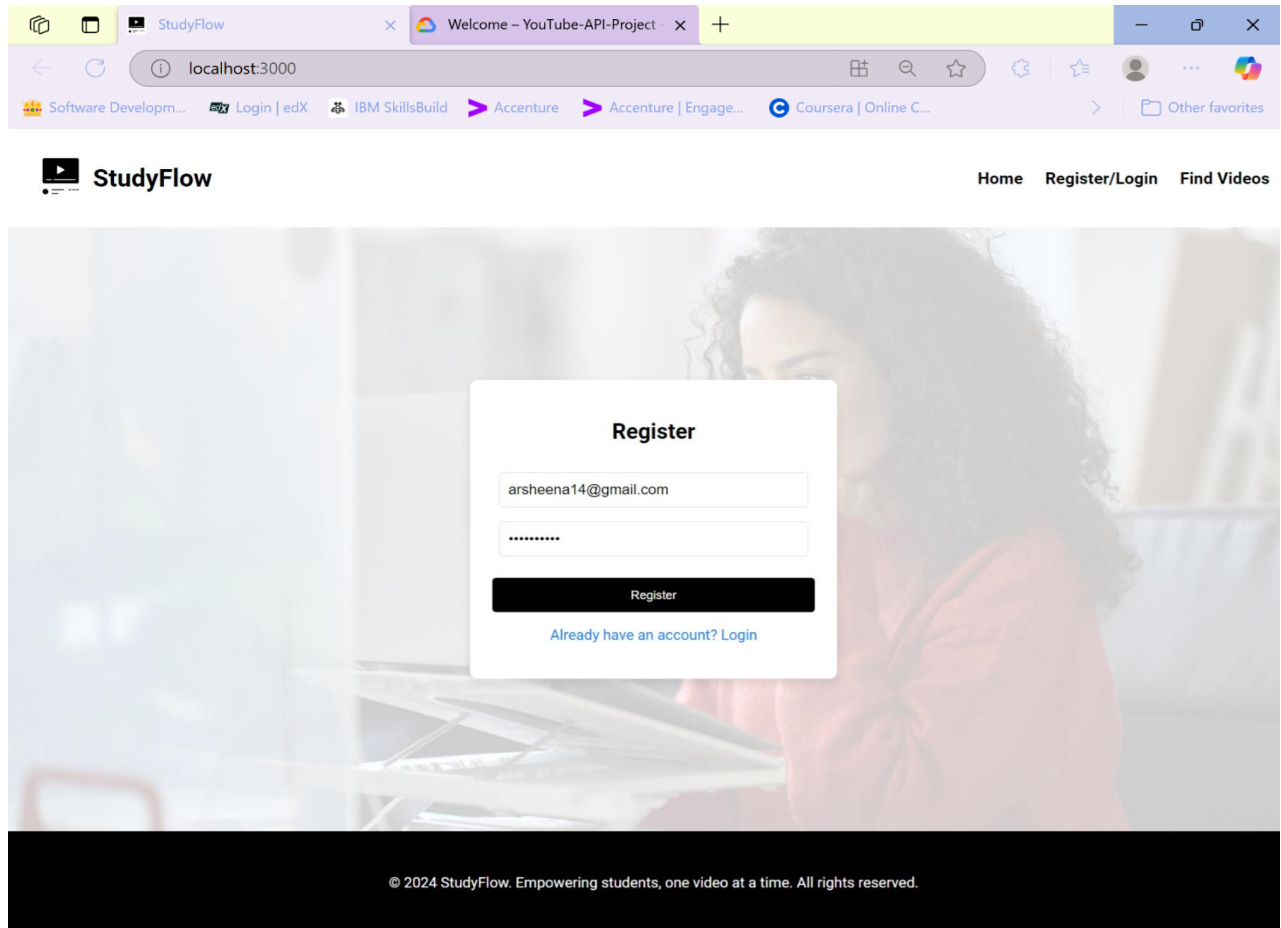
Test ID	Test Name	Input	Expected Output	Actual Output	Status
1	User Registration	Enter valid email and password	Account created successfully	Account created successfully	Pass
2	User Registration (Duplicate Email)	Enter an email which is already registered	Email already exists	Email already exists	Pass
3	User Login	Enter valid credentials – email and password	Redirected to dashboard	Redirected to dashboard	Pass
4	User Login (Invalid Credentials)	Enter invalid credentials	Login Unsuccessful: Invalid credentials entered	Login Unsuccessful: Invalid credentials entered	Pass
5	Syllabus Upload	Upload a valid syllabus image file (E.g: .png)	Topics extracted successfully	Topics extracted successfully	Pass
6	Syllabus Upload (Invalid File)	Upload a non-image file (E.g: .txt)	Invalid file format	Invalid file format	Pass
7	Play Video	Click on play button from the list	Video opens and plays without buffering	Video opens and plays without buffering	Pass
8	Delete Video	Click on delete button from the list	Video gets deleted after asking for confirmation	Video gets deleted after asking for confirmation	Pass

6. SCREENSHOTS

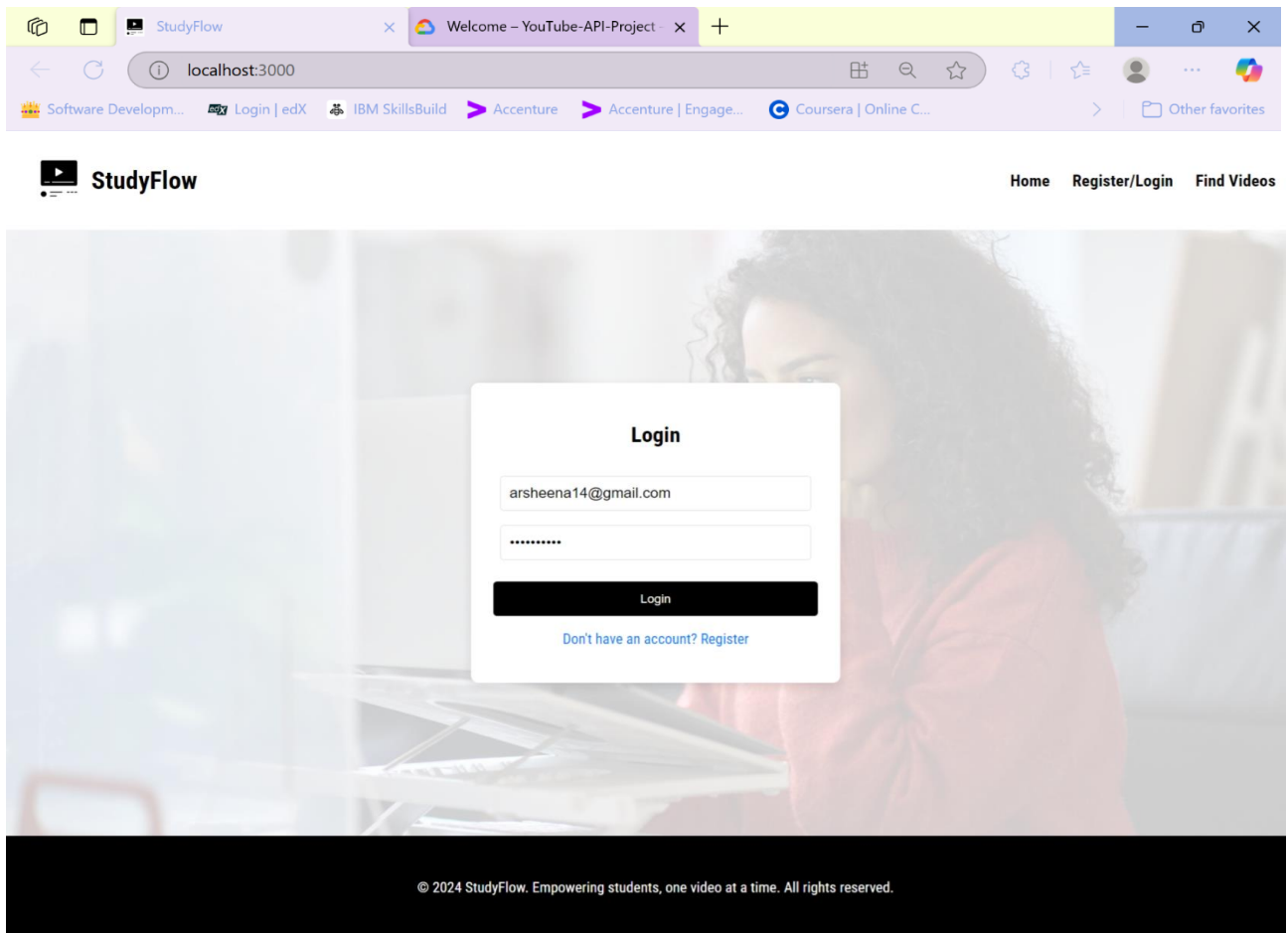
6 Screenshots



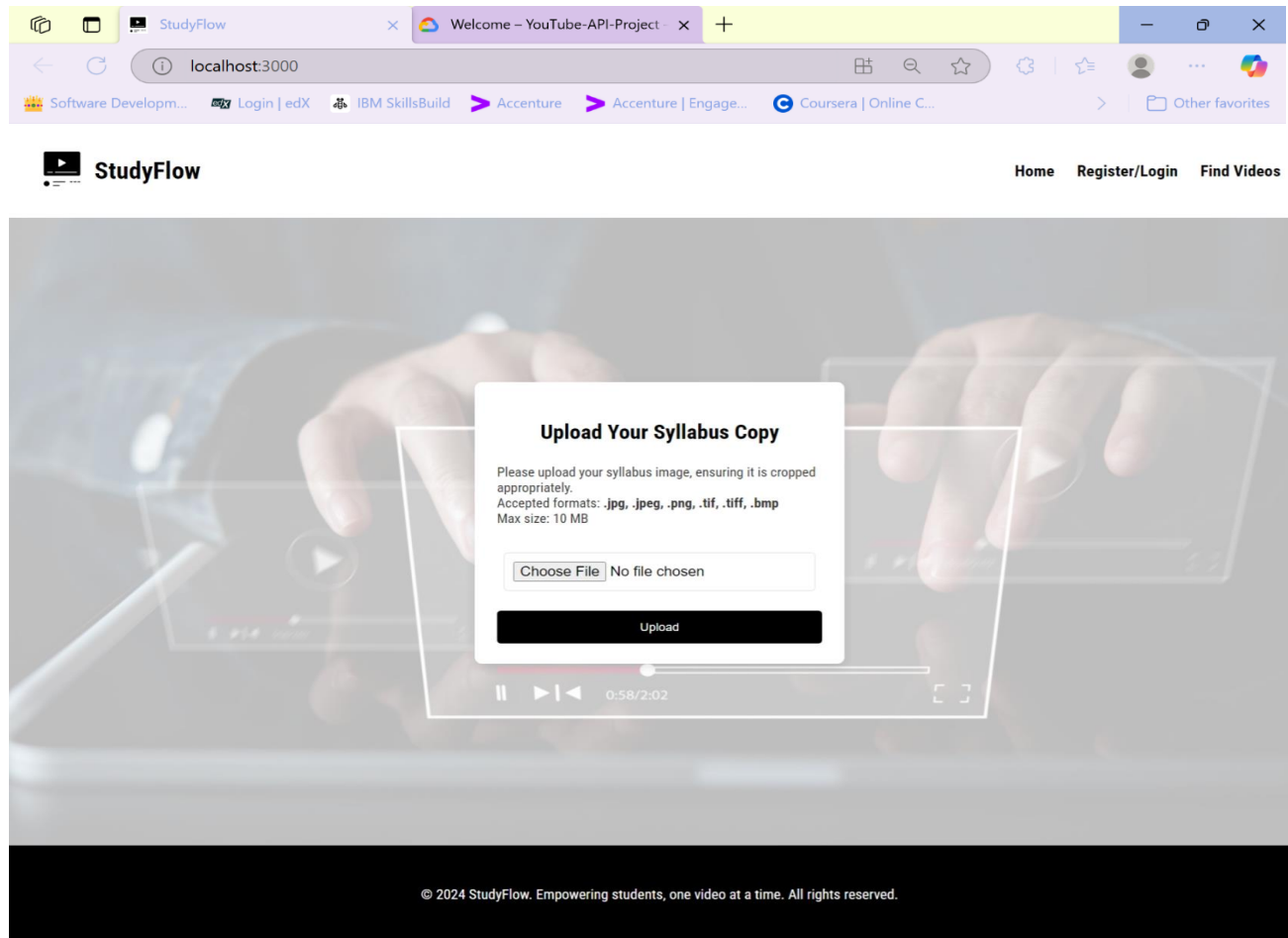
Screenshot 6.1 Main Page



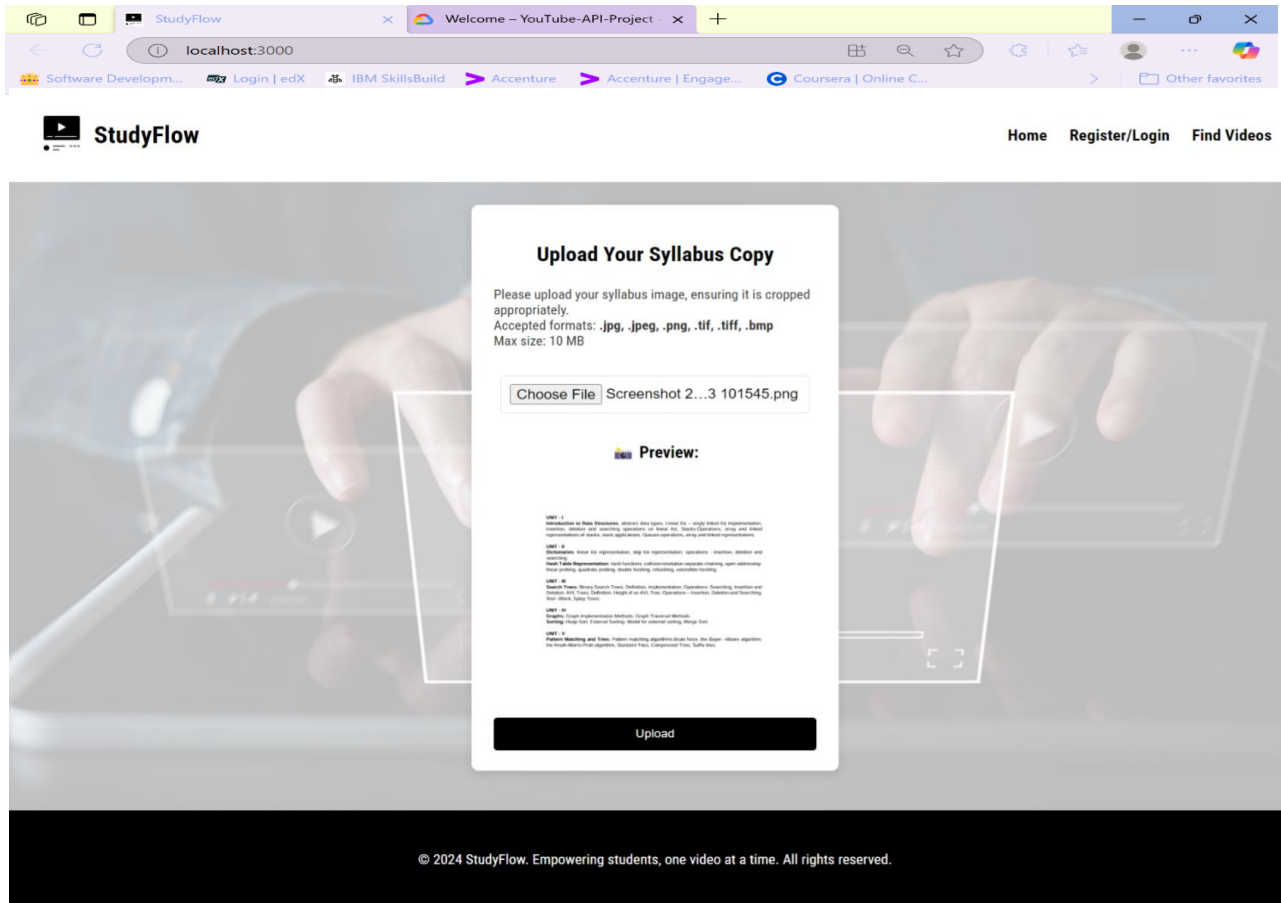
Screenshot 6.2 User Registration



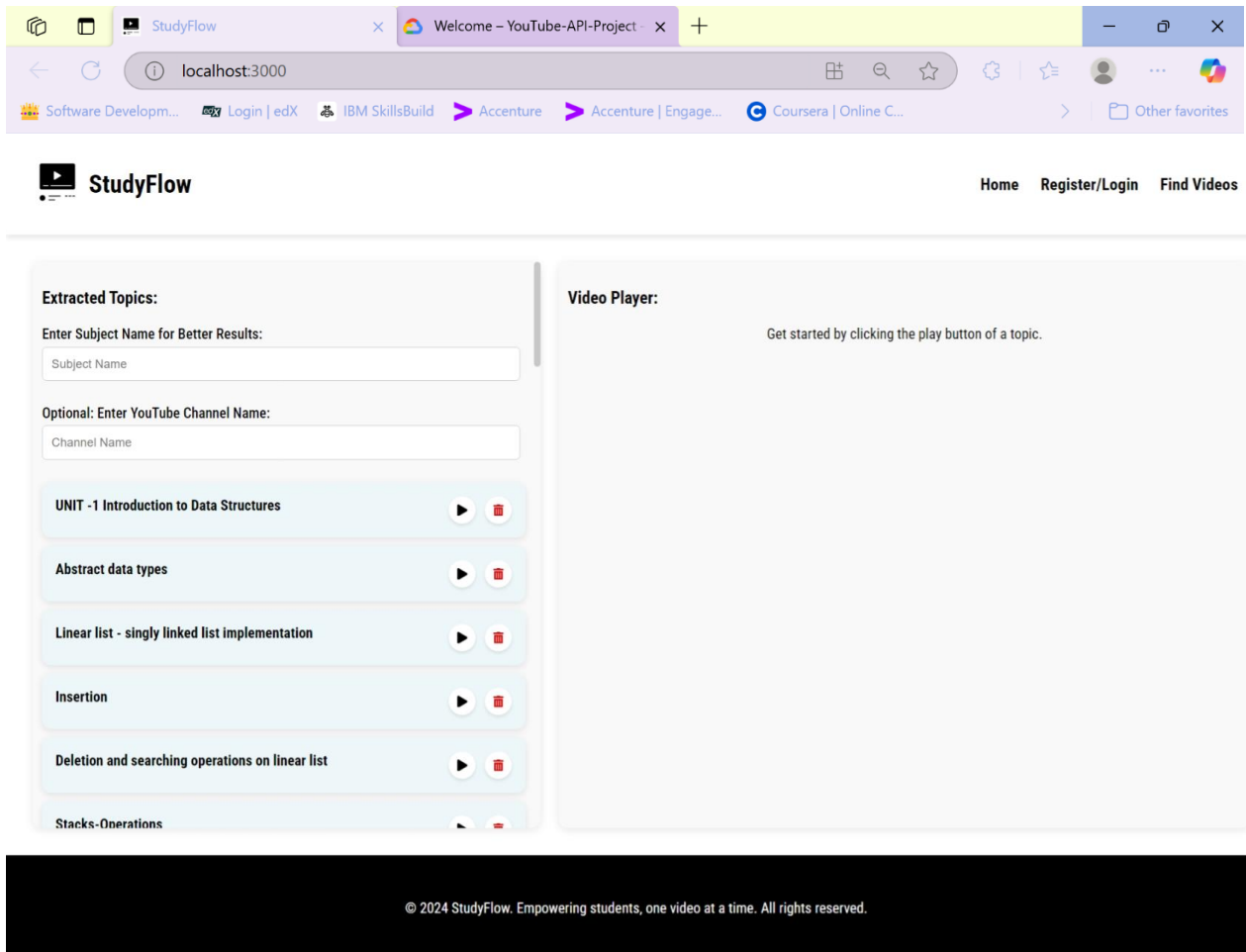
Screenshot 6.3 User Login



Screenshot 6.4 Uploading Syllabus Copy



Screenshot 6.5 Syllabus Copy Preview



Screenshot 6.6 Results Page

The screenshot displays the StudyFlow web application interface. At the top, there is a browser window with tabs for 'StudyFlow' and 'Welcome - YouTube-API-Project'. The address bar shows 'localhost:3000'. Below the browser window, the StudyFlow logo is on the left, and navigation links 'Home', 'Register/Login', and 'Find Videos' are on the right.

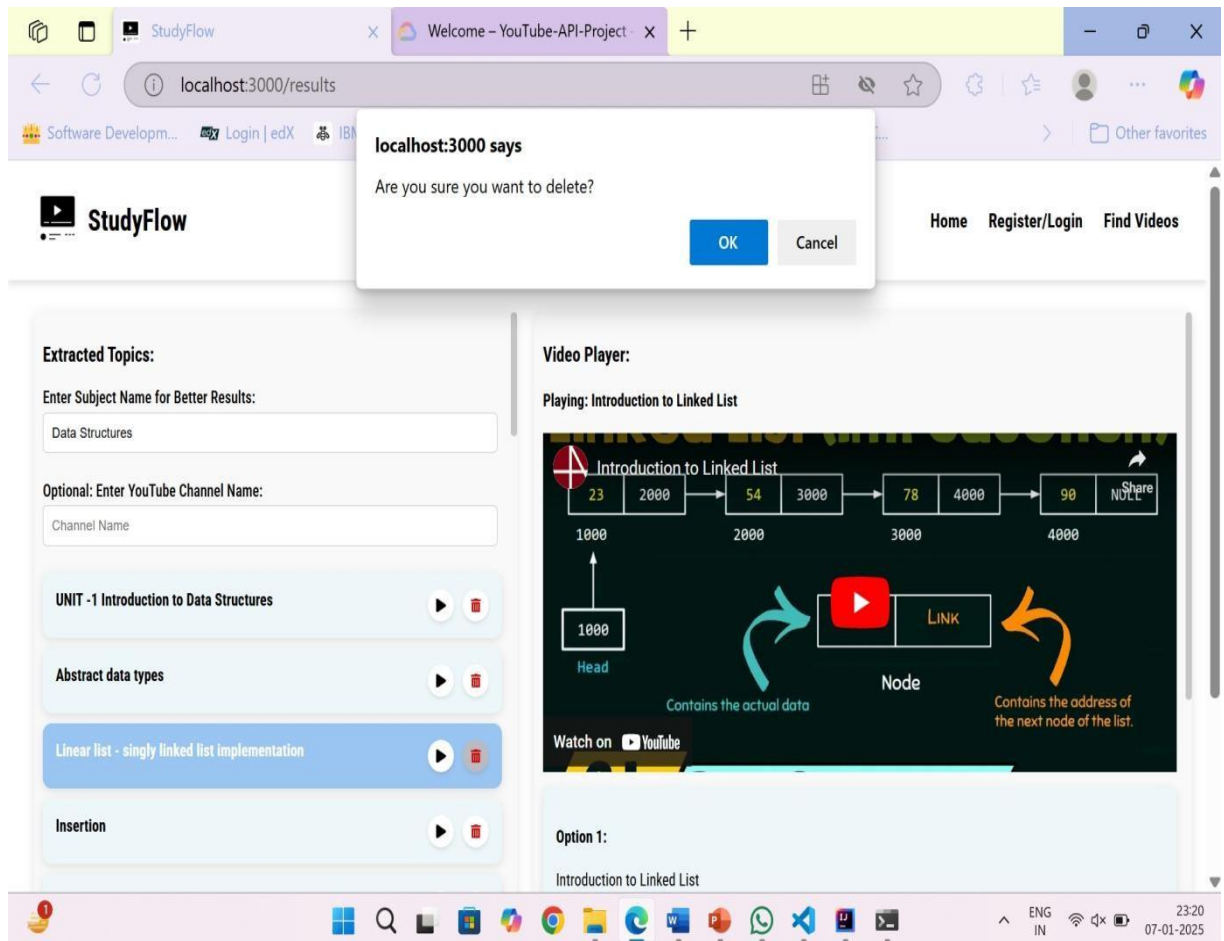
The main content area is divided into two sections:

- Extracted Topics:** This section contains a search bar with the text 'Data Structures' and an optional field for 'Enter YouTube Channel Name:'. Below these are several topic cards, each with a play button and a delete icon. The topics are: 'UNIT -1 Introduction to Data Structures', 'Abstract data types', 'Linear list - singly linked list implementation', 'Insertion', 'Deletion and searching operations on linear list' (highlighted in blue), and 'Stacks-Operations'.
- Video Player:** This section shows a video titled 'Single Linked List (Deleting the First Node)' by 'CodeWithHarry'. The video player includes a play button, a share icon, and a 'Watch on YouTube' link. Below the video player, there is a description of 'Option 1: Operations on Arrays in Data Structures: Traversal, Insertion, Deletion and Searching' and the view count 'Views: 8,07,608'.

The video player also features a diagram of a singly linked list. The list has three nodes: the first node contains '45' and '1000', the second node contains '98' and '3000', and the third node contains '3' and 'NULL'. A red box highlights the first node, and a red arrow points to it with the word 'DELETE!' in red text. The 'head' pointer is shown pointing to the first node.

At the bottom of the page, there is a copyright notice: '© 2024 StudyFlow. Empowering students, one video at a time. All rights reserved.'

Screenshot 6.7 Play Video



Screenshot 6.8 Delete Video

7. CONCLUSION & FUTURE SCOPE

7 Conclusion & Future Scope

7.1 Conclusion

StudyFlow is a web application that helps students find syllabus-aligned educational videos efficiently. By uploading syllabus images, it uses Tesseract OCR to extract text accurately and fetches relevant video content from YouTube, enabling better time management and focused learning.

7.2 Future Scope

- i. Integrate content from platforms like Coursera.
- ii. Use Machine Learning for personalized video suggestions.
- iii. The user will be able to know his learning progress in a subject.

8. REFERENCES

8. References

- [1] R. Mittal and A. Garg, "Text extraction using OCR: A Systematic Review," 2020 Second International Conference on Inventive Research in Computing Applications (ICIRCA), Coimbatore, India, 2020
- [2] H. Fang and M. Bao, "Raw material form recognition based on Tesseract-OCR," 2021 IEEE Conference on Telecommunications, Optics and Computer Science (TOCS), Shenyang, China, 2021
- [3] L. Li and L. Gui, "Design and Development of an Online Education System Based on Spring Boot," 2024 3rd International Conference on Artificial Intelligence and Computer Information Technology (AICIT), Yichang, China, 2024.